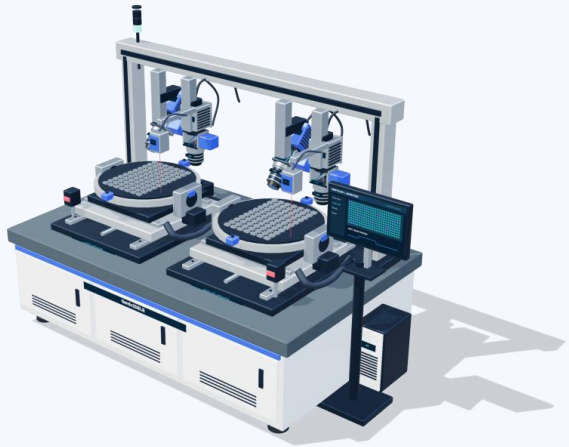
SwiftFlow + HawkeyePlus

Standardized Inspection & Measurement

Architecture, Workflows & Integrated Delivery



SwiftFlow

Equipment, processes and execution

+



HawkeyePlus

Data, models and validation

From hardware design to algorithms and production workflows

2026.10

Presentation Roadmap

Start with a shared method, then explore the two platforms and their handoff

01



Vision & Rationale

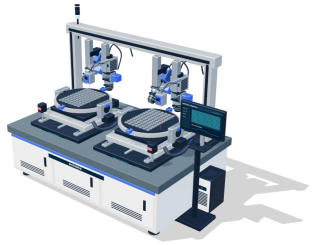
Establish a shared engineering method

Platform roles

Why standardize

03–06

02



SwiftFlow

Equipment, workflows and algorithm execution

Equipment integration

Workflow design

07–10

03



HawkeyePlus

Industrial vision model development and validation

Model development

Model portfolio

11–16

04



Integrated Delivery

From development results to production readiness

Adaptation & acceptance

Shared delivery assets

17–18

Closing: Summary & Discussion 19–20

01 Vision & Rationale

Establish a shared engineering method

How do hardware, workflows and algorithms work as one method?



01.1
Platform roles

01.2
Why standardize

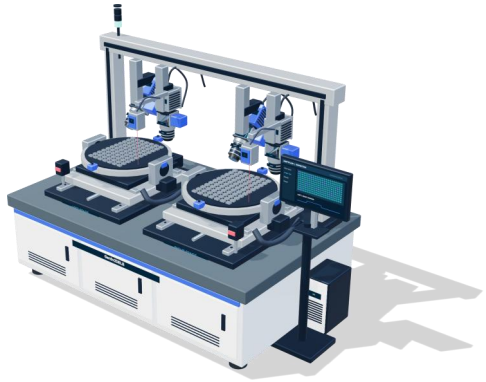
01.3
Standard workflow

SwiftFlow + HawkeyePlus | One shared method, two complementary platforms

One Standard Workflow for Inspection & Measurement

Connect hardware design, inspection workflows and algorithm development through two complementary platforms

SwiftFlow



- Equipment integration
- Workflow orchestration
- Inspection execution

Turn hardware, algorithms and process logic into executable workflows

Samples →

← Models

HawkeyePlus



- Data & annotations
- Model training
- Validation & release

Turn industrial images into validated, deliverable models

Shared method: Define the goal → Design & build → Adapt & validate → Standardize delivery



Inspection

Defect location · Type · Decision



Measurement

Dimensions · Height · Geometry



Traceable delivery

Configuration · Version · Evidence

Why a Shared Engineering Method Matters

Make inputs, deliverables and acceptance criteria clear at every step

Align engineering inputs



Align hardware, imaging and algorithm needs

Common design inputs

- Inspection / measurement goals
- Image quality and calibration

Reuse proven work



Turn proven knowledge into reusable assets

Reusable deliverables

- Device and workflow configuration
- Models and algorithm parameters

Make acceptance explicit



Reproduce results on production equipment

Shared acceptance criteria

- Independent samples and references
- Quality, timing and traceable results

Standardization gives handoffs, reuse and maintenance a shared baseline

Define the method and delivery agreements; use both platforms to execute them

From Engineering Goals to Standardized Delivery

Recommended workflow: clear inputs, engineering responsibilities and reviewable deliverables

01

Define goals



What to inspect or measure

Quality, timing and constraints

Requirements & acceptance

02

Design hardware



Camera, optics and lighting

Motion, sensing and calibration

Hardware design & samples

03

Design workflows



Positioning, capture and timing

Exceptions and result handling

Flow & device configuration

04

Develop algorithms



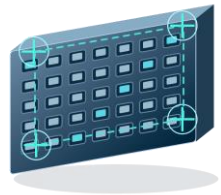
Classical vision / geometry

AI training and validation

Parameters / model package

05

Validate jointly



Site samples and references

Quality, metrology and timing

Acceptance evidence

06

Deliver a baseline



Configuration, models, versions

Traceable results and iteration

Reusable engineering assets

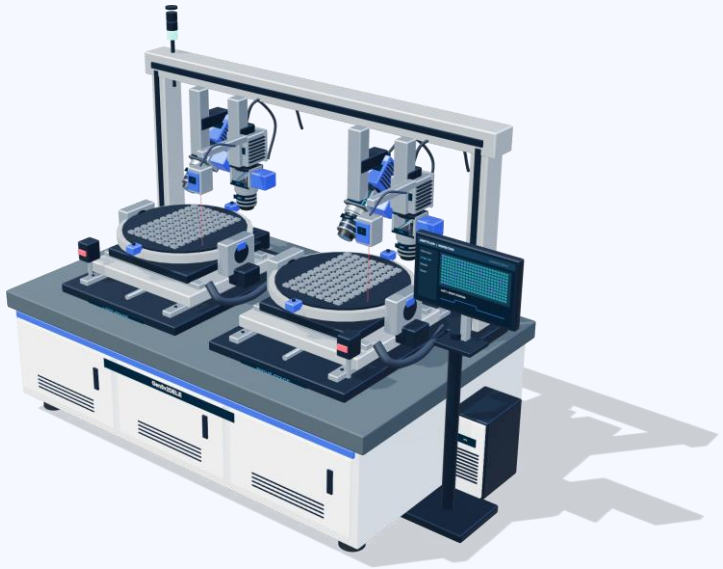
SwiftFlow	Equipment integration, commissioning, calibration, workflow execution and results
HawkeyePlus	Training data, model development, evaluation and model version management

Shared deliverables: hardware + workflow + algorithm baselines + acceptance evidence

02 SwiftFlow

Equipment, workflows and algorithm execution

How can equipment and algorithms become an executable workflow?



02.1

Equipment integration

02.2

Workflow design

02.3

Inspection & measurement

SwiftFlow + HawkeyePlus | One shared method, two complementary platforms

SwiftFlow | Equipment & Process Execution

Use configurable workflows to connect equipment, classical vision, inference and results

Workflow design



Visual authoring

Reusable process flows

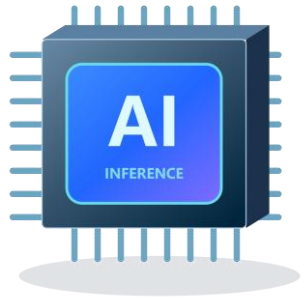
Shared execution engine



Sequential / parallel execution · Shared data

Run control · Failure handling

Algorithms & results



Classical vision + Model inference

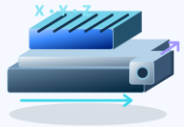
Measurement + Result storage

Unified equipment integration

Standard interfaces · Multiple instances · Vendor adapters



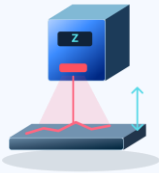
Industrial cameras



Motion control



Controlled lighting



Height sensing



Optics / metrology



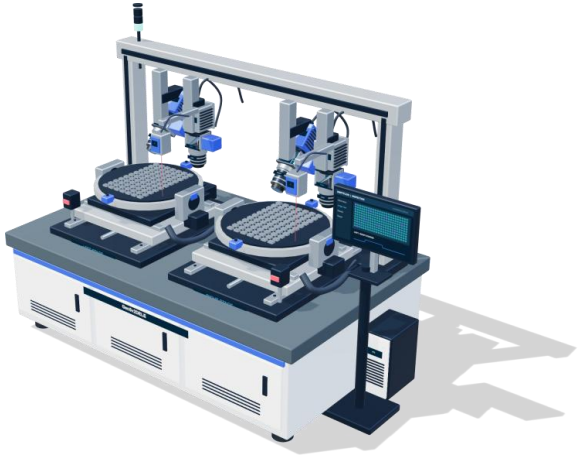
IO / Line communication

Cross-cutting support: logs & alarms · timing · traceability · storage · simulation

From Hardware Design to Executable Workflows

Translate hardware choices, commissioning and process knowledge into reusable configuration and workflows

Engineering design inputs



Camera / Optics / Lighting / Motion / Sensors

Define field of view, resolution, timing and calibration

Configuration & workflow implementation



Map device instances

Commission & calibrate

Reuse parameters & flows



Locate → Capture → Analyze → Decide & record

Deliver reusable engineering assets

Device mappings & configuration · Calibration · Workflows · Process parameters · Result definitions

One Workflow Foundation for Inspection & Measurement

Combine classical vision, AI and metrology as needed, sharing acquisition, orchestration and traceability

Inspection: locate defects and decide

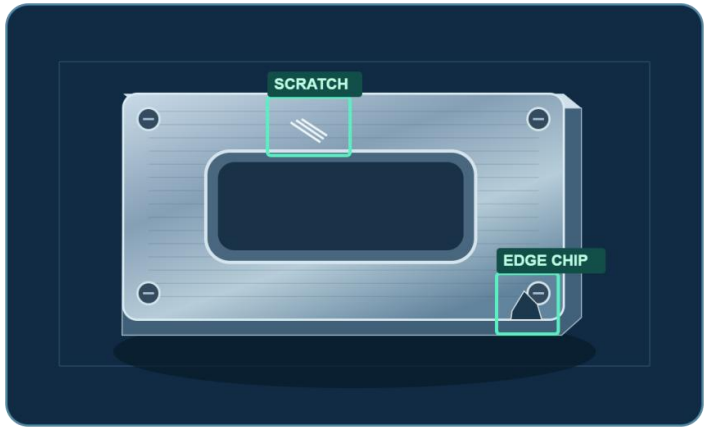
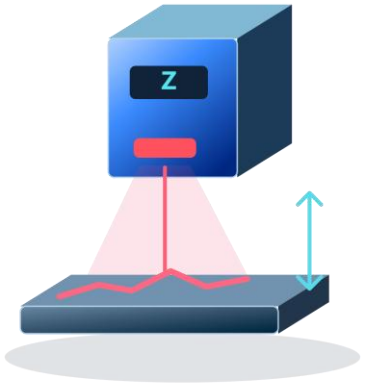


Image → Classical vision / AI → Decision

Output: location, class and pass/fail

Measurement: produce defined metrics



Calibration → Capture / segmentation → Measurement

Output: dimensions, height and geometry

Shared foundation: acquire data → run algorithms → apply rules → retain results

Measurement requires calibration, units and reference validation; AI can assist localization or segmentation

03 HawkeyePlus

Industrial vision model development and validation

How do we turn data into validated, deliverable models?



03.1
Model development

03.2
Model portfolio

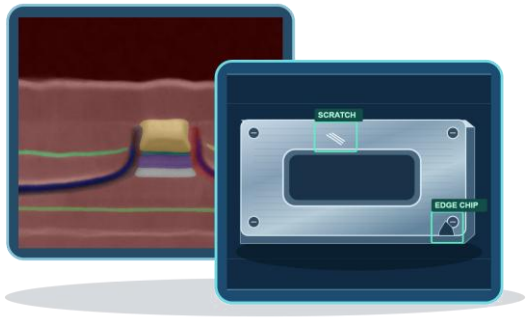
03.3
Validation & versions

SwiftFlow + HawkeyePlus | One shared method, two complementary platforms

HawkeyePlus | Industrial Vision Model Development

Connect industrial images, model training, validation and release in one platform

01 Data & Annotations

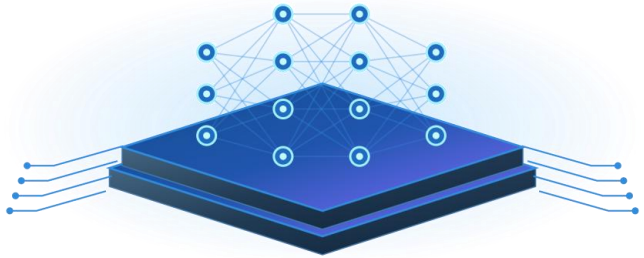


Images + Labels / Masks

Import · Annotate · Preview

Visual Training Hub

Dataset · Model · Training



Job Setup · GPU Selection · Training Monitor

02 Validation & Delivery

Evaluate & Infer
Metrics · Prediction preview

Model Export
Convert · Package · Download

Model Registry
Product versions · Activation

Multi-framework Training

PaddlePaddle
Detect · Classify · Segment

PyTorch
Detect · Segment

anomalib
Anomaly Detection

Compute & Asset Foundation

Compute
GPU / Python

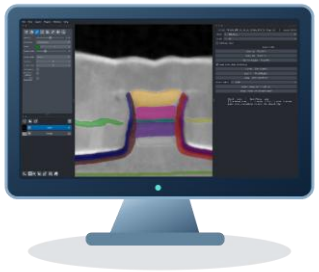
Data & Assets
Images · Weights · Logs

- **Management** Project ownership · User roles · Environment checks · GPU status · Storage quotas · Activity logs

A Visual Lifecycle: From Data to Models

Bring preparation, training and validation into a shared model-development workflow

01 Prepare Data



Import images & labels

Convert formats · Review samples

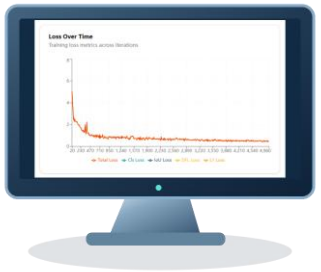
02 Configure



Dataset + Model + Parameters

Reusable templates · Visual setup

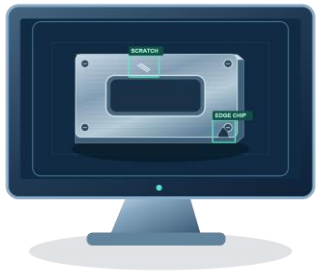
03 Training Monitor



Run jobs · Track training

Progress · Loss · Metrics

04 Validate



Evaluate · Preview predictions

Inspect results · Assess readiness

05 Deliver Models



Export · Register versions

Select the active product model

Use validation results to refine data and configuration

Four vision tasks. One shared training platform.

Object Detection
Locate objects and defects

Image Classification
Recognize categories and quality

Semantic Segmentation
Map regions and boundaries

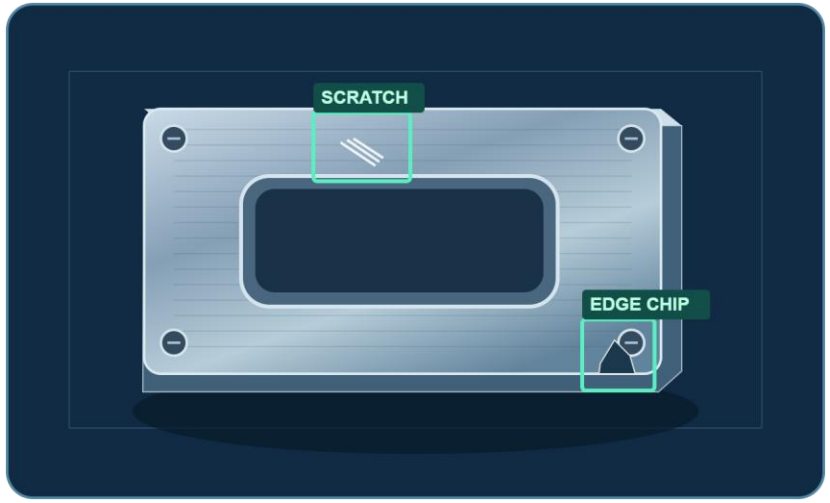
Anomaly Detection
Learn from normal samples

Simplify adoption / Reuse training knowledge / Build model assets

Object Detection | Industrial Defect Localization

Locate defects and identify their classes with bounding boxes and confidence scores

Locate & Classify Defects



Output: Boxes · Classes · Confidence

Illustrative output, not a model prediction

PaddleDetection

7 presets

PP-YOLOE · YOLOv3 · PicoDet · RT-DETR

Faster R-CNN · RetinaNet · CenterNet

TorchDet

4 presets

Faster R-CNN · RetinaNet · FCOS · SSD / SSDLite

SSD variants: VGG16 → SSD300; MobileNetV3 → SSDLite320



COCO Images & Labels

Box labels · Classes



Training & Metric Tracking

mAP / mAP50 · Training curves



Detection Preview

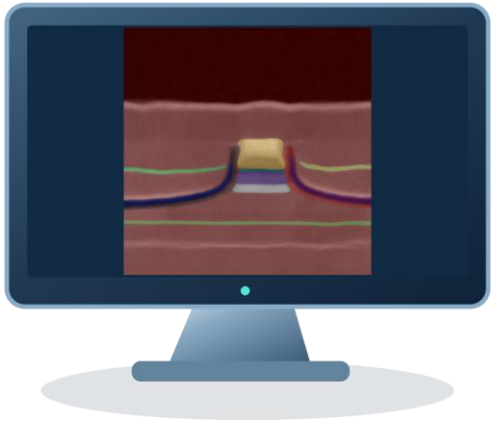
Locate defects · Inspect classes

Presets are backend-specific; PP-YOLOE and RT-DETR are not TorchDet models.

Segmentation Models | Pixel-level Understanding

TEM images illustrate segmentation across compact, encoder-decoder and Transformer architectures

Pixel-level Prediction Example



Regions · Boundaries · Label masks

TEM segmentation example from this project

PaddleSeg

7 presets

PP-LiteSeg · BiSeNetV2 · OCRNet · UNet · DeepLabV3+
SegFormer-B0 · FCN

TorchSeg

4 presets

UNet · DeepLabV3 (UI: DeepLabV3+) · FCN · LR-ASPP

TorchSeg maps DeepLabV3P to torchvision DeepLabV3, not a V3+ decoder.



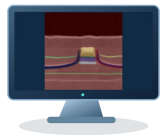
Dataset Layout

Images + masks + train / validation splits



Evaluation Metrics

mIoU · Dice · Accuracy



Applications

Defect regions · Microstructure · Boundaries

Classification & Anomaly Detection

From image-level labels to learning normal appearance, reuse the same model-development workflow

PaddleClas | Image Classification

Learn labeled categories; predict image-level classes



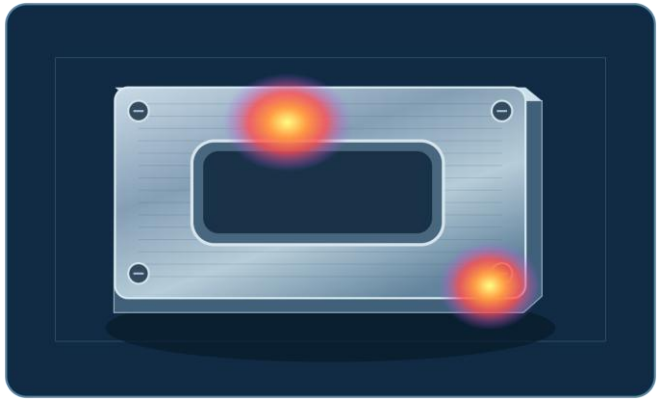
Illustrative classification

9 Classification Variants

ResNet18 · ResNet34 · ResNet50 · ResNet101
MobileNetV3 Small / Large (x1.0)
PP-LCNet x1.0 · PP-HGNet Small
Swin Transformer Tiny (patch4 / window7 / 224)

TorchAnomaly | Anomaly Detection

Learn normal appearance; produce anomaly scores and heatmaps



Illustrative anomaly heatmap

5 anomalib Presets

PatchCore · PaDiM · STFPM · EfficientAD

SuperSimpleNet

Current UI presets only; custom YAML compatibility depends on the selected backend.

04 Integrated Delivery

From development results to production readiness



How do delivery packages and joint validation connect both platforms?

04.1
Adaptation & acceptance

04.2
Shared delivery assets

04.3
Results & version traceability

SwiftFlow + HawkeyePlus | One shared method, two complementary platforms

Connect Development and Production with a Standard Handoff

Adapt and jointly validate model deliverables before using them in inspection or measurement workflows

HawkeyePlus



Validated model & version

→

Adaptation & joint acceptance

- **Match the input**
  Image shape, channels and preprocessing
- **Match the output**
  Labels, thresholds and output decoding
- **Validate on site**
  Independent samples, measurement references and timing

→

SwiftFlow



Inference & decisions in the workflow

Recommended delivery baseline



Hardware baseline

Setup · Calibration · References



Workflow

Steps · Parameters · Rules



Model package

Version · Labels · Thresholds



Acceptance

Samples · Evidence · Records

Build a Reusable Standard Workflow

Organize engineering around a shared method, complementary platforms and a defined delivery baseline

One engineering method



Start with inspection or measurement goals

Define inputs and outputs across disciplines

Two complementary platforms



SwiftFlow connects equipment and execution

HawkeyePlus develops and validates models

A standard delivery baseline



Deliver configuration, models and evidence

Validate on site and retain versions and results

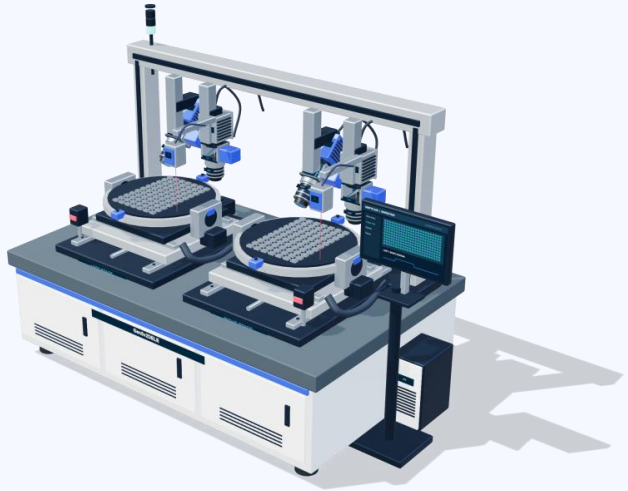
Define the goal → Engineer the solution → Validate jointly → Deliver traceably

The goal: help users deliver inspection and measurement through a consistent engineering approach

Thank You

Questions & Discussion

From hardware design to inspection and measurement delivery



- Which use case should we prioritize?
- How should we define acceptance and delivery?
- Which workflows, models and knowledge can we reuse?



One shared method · Two complementary platforms · Reusable engineering deliverables